

# Week 3 (Thursday, July 10)

## Linear Regression and Classification

**CMPE-257: Machine Learning - Summer 2025**



**SJSU**

**SAN JOSÉ STATE  
UNIVERSITY**

*Instructor: Rafael E Banchs*  
*Email: [rafael.banchs@sjsu.edu](mailto:rafael.banchs@sjsu.edu)*

# Today's plan...

## *Main Topic: Linear Regression and Classification*

**5:30-6:00** Quiz 1

**6:00-7:00** Session 1

**7:00-7:30** Break

**7:30-9:00** Session 2

**9:00-9:30** Term Projects and HW3

### ***Session 1:***

- Linear Regression

### ***Session 2:***

- Linear Classification
- Hands-on coding session 3

**Second Homework is Due Today!**

# QUIZ-TIME

**First Quiz**



**30 minutes**

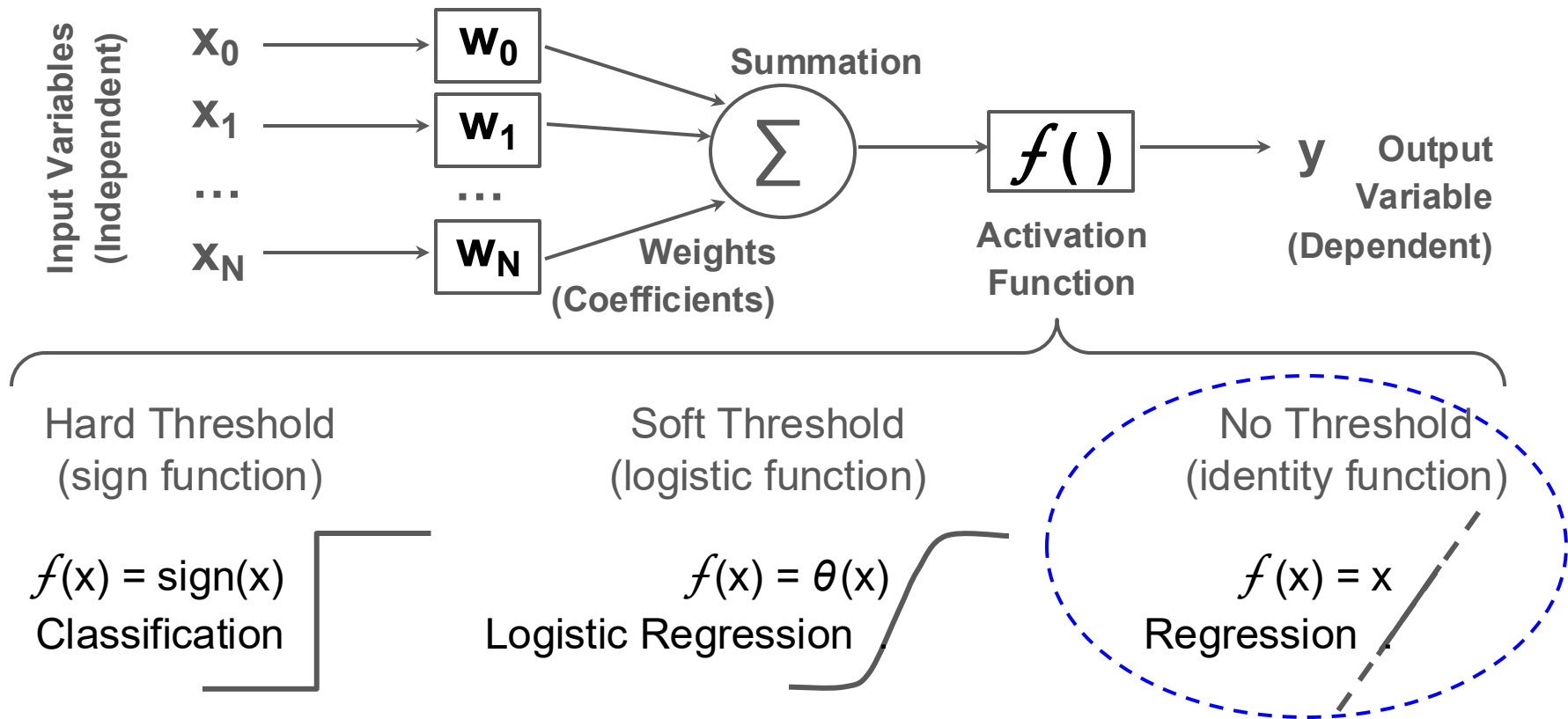
# Week 3 - Session 1

## *Linear Regression*

- Simple Linear Regression
- Multiple Linear Regression
- Linear Model Extensions

# The linear model

$$y = f(\sum w_n x_n)$$



## Properties of a linear function

$$\mathbf{f}(\mathbf{x}) = \sum \mathbf{w}_n \mathbf{x}_n$$

- **Additivity:**

$$f(s+x) = f(s) + f(x)$$

$$f(s+x) = \sum_n w_n (s_n + x_n) = \sum_n w_n s_n + \sum_n w_n x_n = f(s) + f(x)$$

- **Homogeneity:**

$$f(ax) = a f(x)$$

$$f(ax) = \sum_n w_n a x_n = a \sum_n w_n x_n = a f(x)$$

## Simple linear regression

$$y = \cancel{w_0} \overset{b}{\cancel{x_0}^1} + \cancel{w_1} \overset{m}{\cancel{x_1}^x}$$

- For a given set of observations:  $\Rightarrow y = b + m x$

$$(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), \dots (x_N, y_N)$$

- We want to minimize the Mean Square Error (MSE)

$$\text{MSE} = 1/N \sum_n (y_n - m x_n - b)^2$$

- Optimal values for  $m$  and  $b$  are obtained by solving the following equations:  $\delta \text{MSE} / \delta m = 0$  and  $\delta \text{MSE} / \delta b = 0$



## Simple linear regression solution

- Optimal values:

$$\hat{m} = \frac{\sum (x - \bar{x}) (y - \bar{y})}{\sum (x - \bar{x})^2}$$

$$\hat{b} = \bar{y} - \hat{m} \bar{x}$$

- Where:

$$\bar{x} = 1/N \sum x \quad \text{and} \quad \bar{y} = 1/N \sum y$$





## Proposed exercise...

- Prove that previous class results:

$$m = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}$$

$$b = 1/N (\sum y - m \sum x)$$

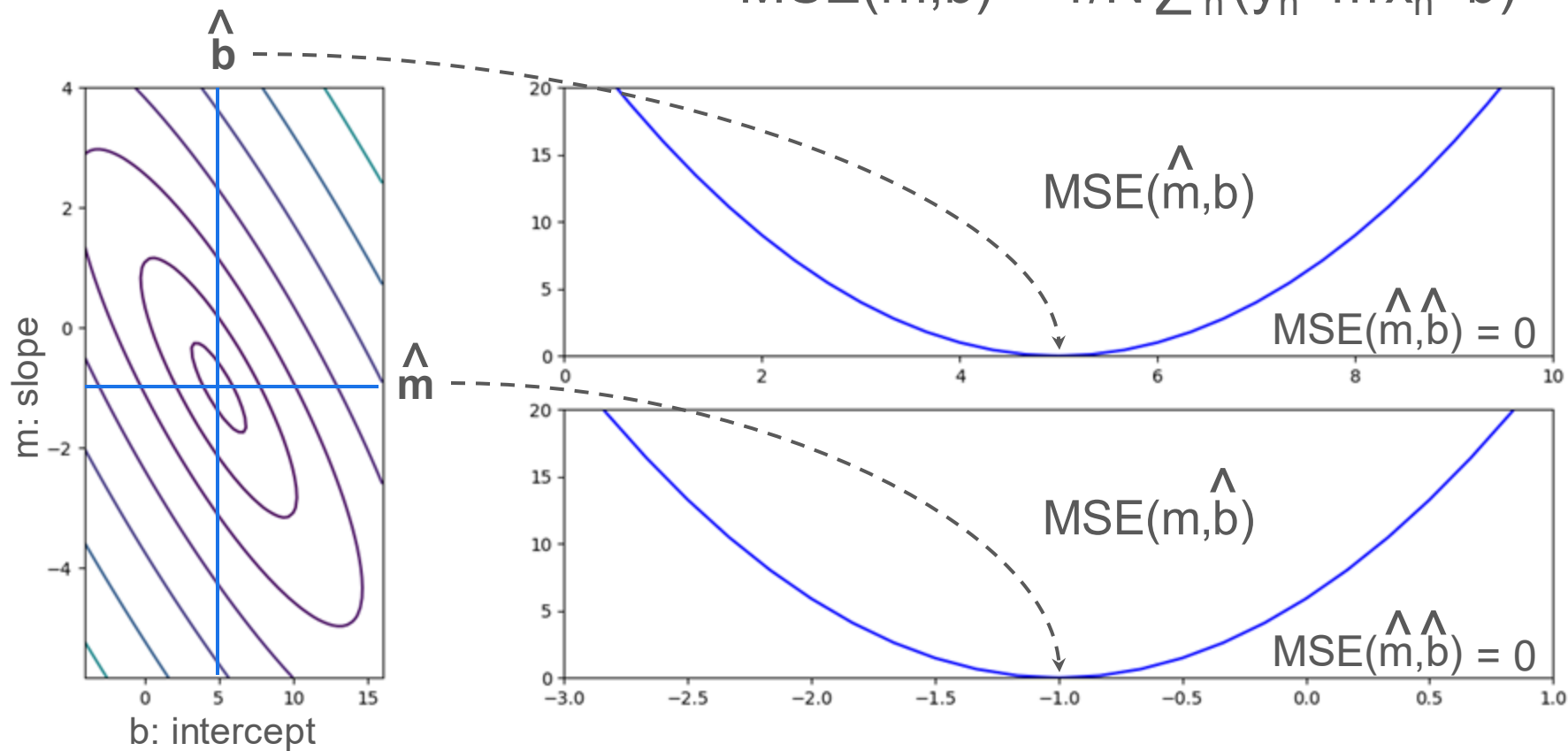
- Are exactly the same as:

$$\hat{m} = \frac{\sum (x - \bar{x}) (y - \bar{y})}{\sum (x - \bar{x})^2}$$

$$\hat{b} = \bar{y} - \hat{m} \bar{x}$$

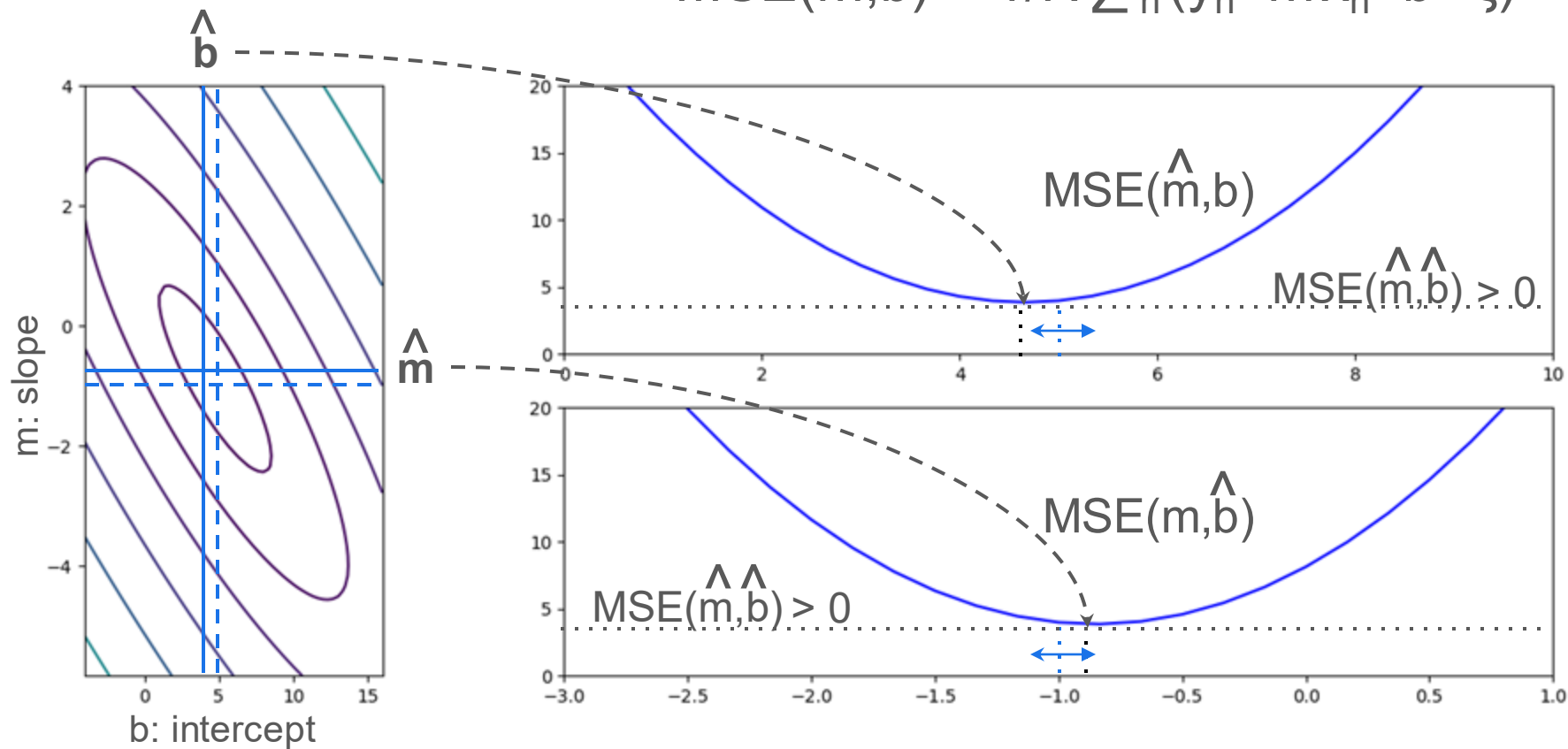
# The error surface

$$\text{MSE}(m,b) = 1/N \sum_n (y_n - mx_n - b)^2$$



# The effect of noise

$$\text{MSE}(m,b) = 1/N \sum_n (y_n - mx_n - b - \xi)^2$$



## Standard error computation

- How much the estimated parameters  $\hat{m}$  and  $\hat{b}$  differ from the actual ones, on average?

$$SE(\hat{m})^2 = \frac{\text{var}(\xi)}{\sum (x - \bar{x})^2} \quad SE(\hat{b})^2 = \text{var}(\xi) \left( \frac{1}{N} + \frac{\bar{x}^2}{\sum (x - \bar{x})^2} \right)$$

- Assumptions: additive noise values  $\xi$  for each observation are uncorrelated and have common variance  $\text{var}(\xi)$

## Residual standard error (RSE)

- In practice,  $\text{var}(\xi)$  is unknown, but can be estimated from data

- We define the residual sum of squares (RSS) as:

$$\text{RSS} = N \times \text{MSE}(\hat{m}, \hat{b}) = \sum (y - \hat{m}x - \hat{b})^2 = \sum (y - mx - b - \xi)^2$$

- and the residual standard error (RSE) as:

$$\text{RSE} = \sqrt{\text{RSS} / (N - M)}$$

where  $M$  is the number of model parameters  $|\{b, m\}|$

- Then,  $\text{var}(\xi)$  is estimated as  $\text{var}(\xi) \approx \text{RSE}^2$



## Confidence intervals

- Standard errors allow for computing confidence interval
- $\beta\%$  confidence interval: range of values that contains the true unknown value of the parameter with  $\beta/100$  probability
- For linear regression, the 95% confidence interval for a parameter  $w$  is approximately  $[\hat{w} - 2 \text{ SE}(\hat{w}), \hat{w} + 2 \text{ SE}(\hat{w})]$

$$\mathbf{95\% \text{ Conf. Int. for } w \approx \hat{w} \pm 2 \text{ SE}(\hat{w})}$$



## Confidence interval exercise

- Consider the dataset  $d = \{ p_1:(1, 9/2), p_2:(2, 5/2), p_3:(3, 2) \}$
- Verify that the best linear model is  $g = \{ b: 11/2, m: -5/4 \}$
- Compute the residual standard error (RSE)
- Compute the standard error (SE) for each model parameter
- Provide the 95% confidence interval for each model parameter
- Verify whether the computed confidence intervals contain the actual parameter values. The actual function is  $f = \{ b: 5, m: -1 \}$



## Best linear model computation

- Consider the dataset  $d = \{ p_1:(1, 9/2), p_2:(2, 5/2), p_3:(3, 2) \}$
- Verify that the best linear model is  $g = \{ b: 11/2, m: -5/4 \}$

$$\bar{x} = \frac{1}{3} \sum (1+2+3) = 2 \qquad \bar{y} = \frac{1}{3} \sum (9/2+5/2+2) = 3$$

$$\hat{m} = \frac{\sum (x - \bar{x}) (y - \bar{y})}{\sum (x - \bar{x})^2} = \frac{-1 \times 3/2 + 0 \times (-1/2) + 1 \times (-1)}{(-1)^2 + 0^2 + (-1)^2} = -5/4$$

$$\hat{b} = \bar{y} - \hat{m} \bar{x} = 3 - (-5/4) \times 2 = 11/2$$





# Standard error computations

- Compute the residual standard error (RSE)
- Compute the standard error (SE) for each model parameter

$$RSS = (9/2 + 5/4 \times 1 - 11/2)^2 + (5/2 + 5/4 \times 2 - 11/2)^2 + (2 + 5/4 \times 3 - 11/2)^2$$

$$RSS = (1/4)^2 + (-1/2)^2 + (1/4)^2 = 2/16 + 1/4 = 6/16 = 0.375$$

$$RSE = (6/16 / (3-2))^{1/2} = 1/4 \cdot 6^{1/2} = 0.6124$$

$$SE(\hat{m}) = (6/16 \times 1/2)^{1/2} = (3/16)^{1/2} = 0.4330$$

$$SE(\hat{b}) = (6/16 (1/3 + 4/2))^{1/2} = (14/16)^{1/2} = (7/8)^{1/2} = 0.9354$$



# Confidence interval computations

- Provide the 95% confidence interval for each model parameter
- Verify whether the computed confidence intervals contain the actual parameter values. The actual function is  $f = \{ b: 5, m: -1 \}$

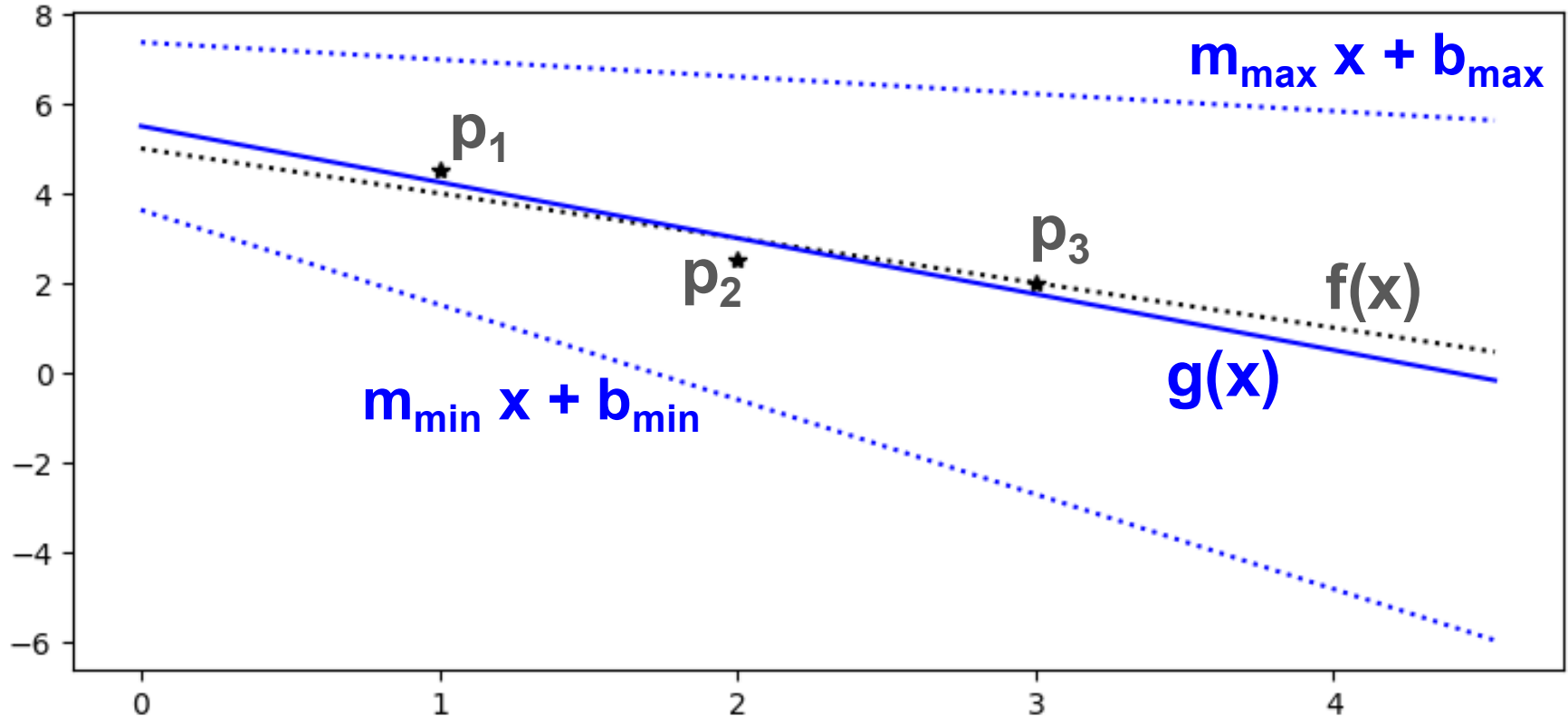
95% Conf. Int. for  $w \approx \hat{w} \pm 2 \text{ SE}(\hat{w})$

$\Rightarrow$  95% CI for  $m \approx -1.25 \pm 2 \times 0.4330 = [ -2.116, -0.384 ]$  ✓

$\Rightarrow$  95% CI for  $b \approx 5.50 \pm 2 \times 0.9354 = [ 3.629, 7.371 ]$  ✓



# Regression result and confidence intervals





# Statistical significance (t-statistics and p-values)

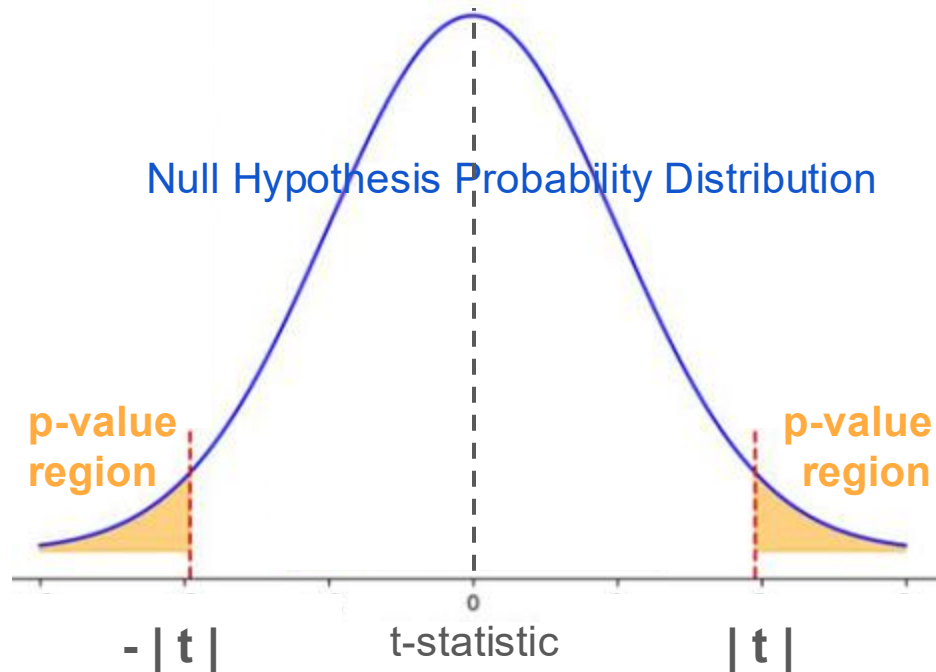
- Null Hypothesis:  $H_0 : \mathbf{w} = \mathbf{0}$  (no relationship between x and y)

- t-statistic:  $t = \frac{\hat{w} - 0}{SE(\hat{w})}$

- p-value =  $P ( |T| \geq t \mid H_0 )$

with p-value < 0.01

$H_0$  can be discarded!





# Statistical significance exercise

- Consider both  $\hat{m}$  and  $\hat{b}$  estimates from the previous exercise:
  - Compute the associated t-statistics
  - Compute the corresponding p-values
- What can be concluded about the statistical significance of the computed linear regression?

Note: you might want to use an online p-value calculator...

<http://courses.atlas.illinois.edu/spring2016/STAT/STAT200/pt.html>



# Statistical significance exercise

- t-statistic and p-value computation:

$$t_{\hat{m}} = \hat{m} / SE(\hat{m}) = -1.25 / 0.4330 = -2.8868 \text{ (p-value} = 0.2123\text{)*}$$

$$t_{\hat{b}} = \hat{b} / SE(\hat{b}) = 5.5 / 0.9354 = 5.8798 \text{ (p-value} = 0.1072\text{)*}$$

- What can be concluded about the statistical significance?  
Because both p-values  $> 0.01$  the null hypothesis cannot be confidently rejected, i.e. results are not statistically significant

\* degrees of freedom in this exercise = 1 (3 data points - 2 parameters)



## Evaluating overall model accuracy

- Overall quality metrics for a linear regression fit:

- Residual Standard Error:  $RSE = \sqrt{RSS / (N - M)}$

$$RSS = \sum (y - \hat{y})^2 = \sum (y - \hat{m}x - \hat{b})^2 \text{ Residual Sum of Squares}$$

$$TSS = \sum (y - \bar{y})^2 = \sum (y - 1/N \sum y)^2 \text{ Total Sum of Squares}$$

- R-squared statistic:  $R^2 = 1 - RSS / TSS$
- F-statistic:  $F = (TSS - RSS) \times (N-M) / (RSS \times (M-1))$

# Model accuracy metric interpretations

- Residual standard error:
  - Measures the misfit between model and data
  - Small values are good, but depend on variable units!
- R-squared statistic:
  - The proportion of output variability explained by the input
  - Close to one implies explainability, close to zero do not
- F-statistic:
  - Tests for an  $H_0$  where all regression coefficients are zero
  - Large values are good, close to one suggest no relations



# Adjusted R-squared statistic

- R-squared does not factor in the number of predictors used
  - Scores are not comparable among models with different number of predictors!
- Adjusted R-squared factors in the number of predictors:
  - Scores across models with different number of predictors are comparable

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2) \times (N - 1)}{(N - M)}$$



## Overall fit accuracy exercise

- Consider the linear regression from the previous exercise:
  - Compute R-squared, Adjusted R-squared and F statistics

$$RSS = 3/8, \quad TSS = (9/2-3)^2 + (5/2-3)^2 + (2-3)^2 = 7/2$$

$$R^2 = 1 - (3/8) / (7/2) = 1 - 3/28 = 25/28 = 0.8929$$

$$\text{Adjusted } R^2 = 1 - 2 (1-25/28) / (3-2) = 11/14 = 0.7857$$

$$F\text{-statistic} = (7/2-3/8) \times (3-2) / (3/8 \times (2-1)) = 25/3 = 8.3333$$

# Multilinear regression

$$\mathbf{y} = \sum \mathbf{w}_n \mathbf{x}_n$$

- For a single data point  $(1, x_1, x_2, \dots, x_p, y)$ :

$$y = w_0 \overset{1}{x_0} + w_1 x_1 + w_2 x_2 + \dots + w_p x_p + \xi = \sum w x + \xi = \mathbf{w}^T \mathbf{x} + \xi$$

there are  $p$  different predictors and  $M=p+1$  model parameters  
 $\mathbf{w}$  and  $\mathbf{x}$  are weight and single data point vectors

- For a collection of  $N$  data points (in-sample dataset):

$$\mathbf{y} = w_0 \mathbf{x}_0 + w_1 \mathbf{x}_1 + w_2 \mathbf{x}_2 + \dots + w_p \mathbf{x}_p + \boldsymbol{\xi} = \mathbf{w}^T \mathbf{X} + \boldsymbol{\xi}$$

where  $\mathbf{X}$  is a  $M \times N$  data matrix of  $M$  columns and  $N$  rows



## Multilinear regression solution

- Similar to simple regression, the in-sample MSE is defined as:

$$\text{MSE}(\hat{\mathbf{w}}) = 1/N \sum (y_n - \hat{\mathbf{w}}^T \mathbf{x}_n)^2 = 1/N \|\mathbf{y} - \mathbf{X} \hat{\mathbf{w}}\|^2$$

- Where MSE can be minimized to obtain the optimal  $\mathbf{w}$
- After using some matrix algebra, the gradient of the in-sample MSE can be computed and made equal to zero:

$$\nabla \text{MSE}(\hat{\mathbf{w}}) = 2/N (\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} - \mathbf{X}^T \mathbf{y}) = 0 \Rightarrow \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{y}$$

- And the optimal weights can be obtained:  $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

## Pseudo-inverse and the hat matrix

- The matrix  $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$  is referred to as the pseudo-inverse of  $\mathbf{X}$

$$\mathbf{X} \hat{\mathbf{w}} = \mathbf{y} \Rightarrow \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{y} \Rightarrow \hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \Rightarrow \hat{\mathbf{w}} = \text{p-inv}(\mathbf{X}) \mathbf{y}$$

Notice that  $\mathbf{X}^T \mathbf{X}$  is typically invertible because  $N \gg M$  and at least  $M$  linearly independent  $\mathbf{x}$  vectors are needed

- If we use  $\hat{\mathbf{w}}$  and  $\mathbf{X}$  to estimate the output  $\hat{\mathbf{y}}$ :

$$\hat{\mathbf{y}} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \Rightarrow \hat{\mathbf{y}} = \mathbf{H} \mathbf{y} \quad \text{with } \mathbf{H} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

Matrix  $\mathbf{H}$  is called the hat matrix because it “puts a hat” on  $\mathbf{y}$



## Hat matrix cool property exercise

- Consider the hat matrix  $\mathbf{H} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$
- Verify that  $\mathbf{H}^2 = \mathbf{H}$  if  $\mathbf{H}$  is a hat matrix

$$\begin{aligned}\mathbf{H}^2 &= \mathbf{H} \mathbf{H} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \\ &= \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T \mathbf{X}) (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \\ &= \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H}\end{aligned}$$

# Model variable selection

- In practical scenarios not all variables are good predictors
- Variable selection can be done by brute force for small number of predictors  $p$  (i.e. comparing all  $2^p$  different possible models)
- For large  $p$ , a variable selection strategy is needed:
  - Forward: from null model, add one predictor at a time
  - Backward: from full model, remove one predictor at a time
  - Mixed: combines forward and backward selection strategies

## Linear model extensions: variable interactions

- Variable interactions can alter (boost or lessen) the output
- An interaction term can be used to account for such effect:

$$\begin{aligned} \mathbf{y} &= w_0 + w_1 \mathbf{x}_1 + w_2 \mathbf{x}_2 + w_3 (\mathbf{x}_1 \times \mathbf{x}_2) + \xi \\ &= w_0 + (w_1 + w_3 \mathbf{x}_2) \times \mathbf{x}_1 + w_2 \mathbf{x}_2 + \xi \end{aligned}$$

denotes element-wise multiplication

- Notice this relaxes the additive assumption (i.e. the association between  $\mathbf{x}_1$  and  $\mathbf{y}$  depends on  $\mathbf{x}_2$ )
- Still a linear regression, but with a new added variable ( $\mathbf{x}_1 \times \mathbf{x}_2$ )



# Linear model extensions: non-linear relationships

- Non-linear dependencies between a variable  $\mathbf{x}$  and the output  $\mathbf{y}$  can also be taken into account
- A non-linear term can be introduced to this effect:

$$\mathbf{y} = w_0 + w_1 \mathbf{x}_1 + w_2 \sigma(\mathbf{x}_1) + \xi$$

denotes a non-linear function

- Notice this relaxes the homogeneity assumption (i.e. the association between  $\mathbf{x}_1$  and  $\mathbf{y}$  is non-linear)
- Still a linear regression, but with a new added variable  $\sigma(\mathbf{x}_1)$

# Things to be aware of when doing linear regressions

- Two or more predictor variables are correlated (collinearity)
- Error terms are correlated and/or of non-constant variance
- Non-linearity of response-predictor relationships
- Synergy effect among variables (i.e. variable interactions)
- Outliers and high-leverage data points

*Despite its limitations and the ubiquity of some of these problems, linear regression continues to be a powerful tool*



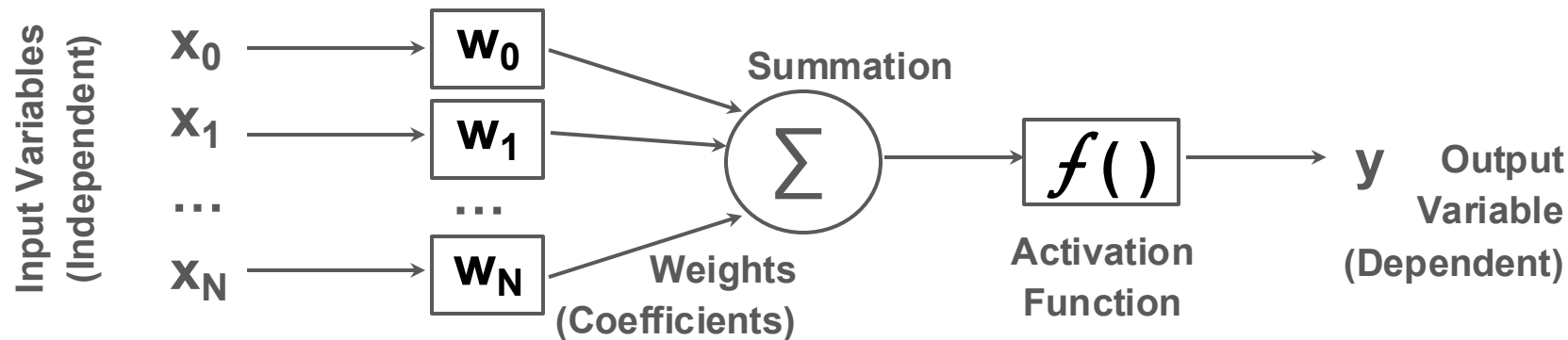
## Week 3 - Session 2

# *Linear Classification*

- Logistic Regression
- Gradient Descent Algorithm
- The Naive Bayes Classifier

# The linear model

$$y = f(\sum w_n x_n)$$



Hard Threshold  
(sign function)

$$f(x) = \text{sign}(x)$$

Classification

Soft Threshold  
(logistic function)

$$f(x) = \theta(x)$$

Logistic Regression

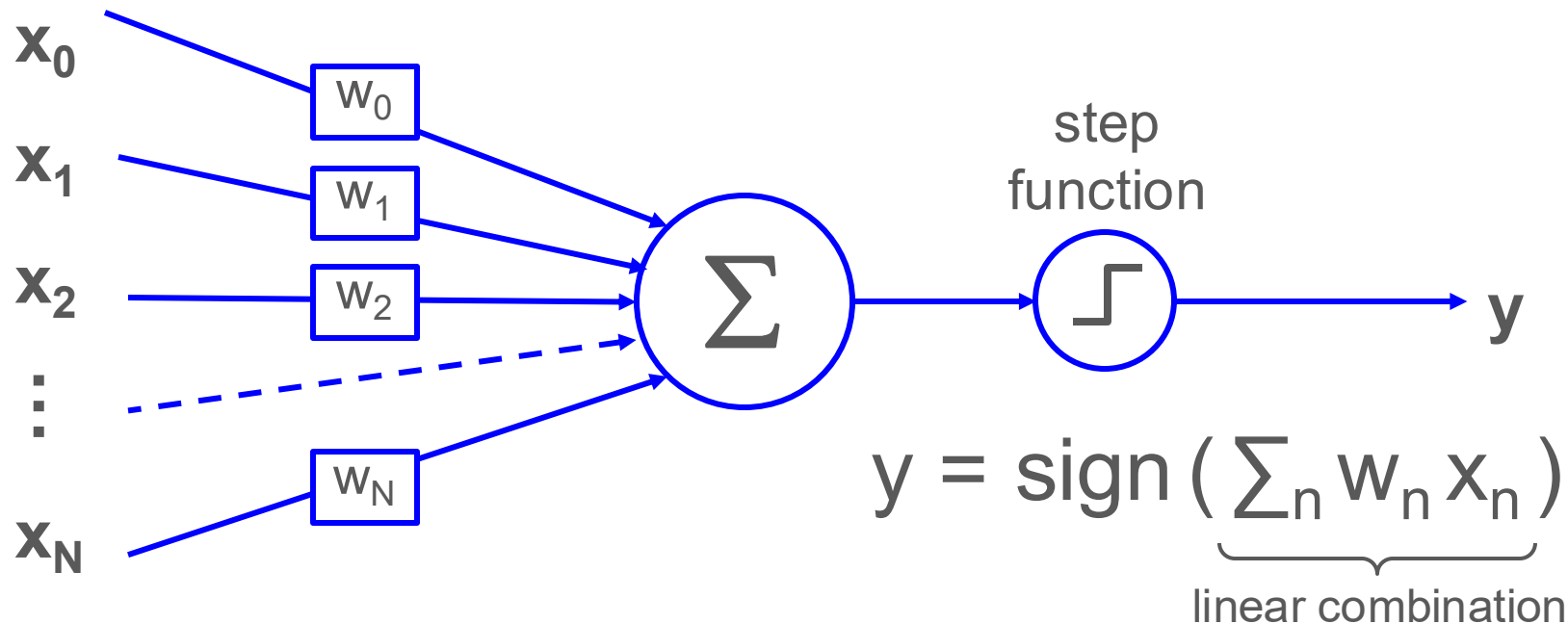
No Threshold  
(identity function)

$$f(x) = x$$

Regression

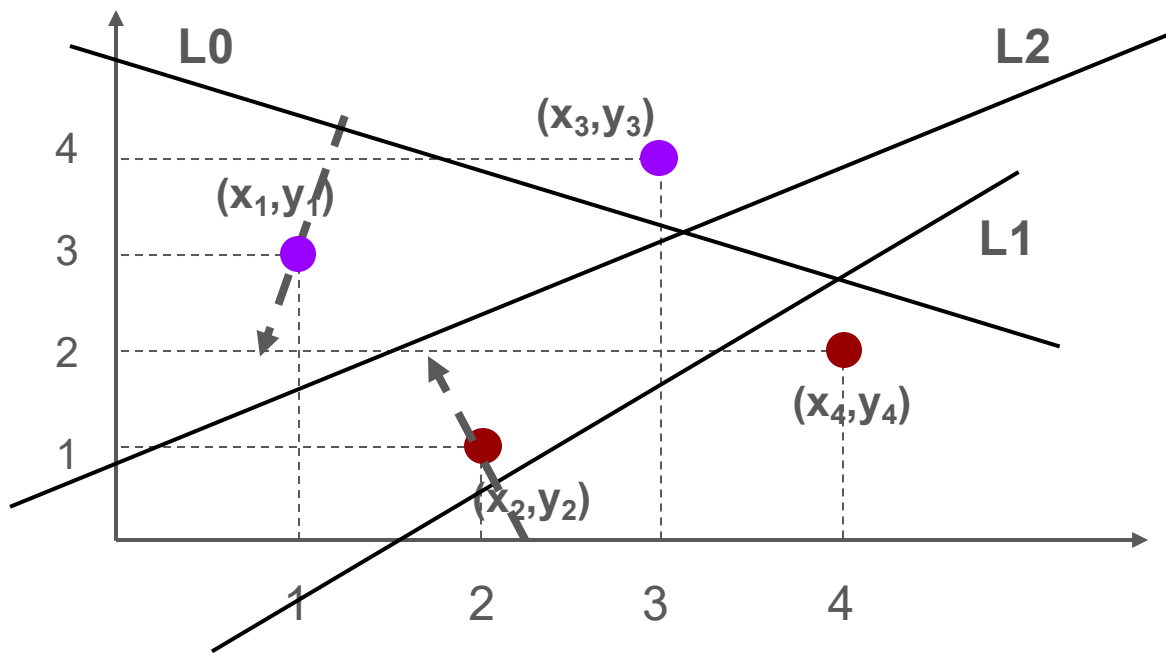
Recall, from week 1...

## *The perceptron as a classifier*



Recall, from week 1...

## *The perceptron learning algorithm (PLA)*



Keep updating until  
all data points are  
correctly classified  
(i.e.  $e_k = 0$  for all  $k$ )

What happens if  
the classes are not  
linearly separable?

## Disadvantages of using a hard threshold

- If classes are not separable the PLA keeps updating until reaching maximum number of iterations
  - There is not guarantee of ending on a good solution
  - Pocket algorithm (PKA): keep best solution encountered
  - PKA does not guarantees error minimization either
- Minimizing the discrete error function  $1/N \sum_N | \text{sign}(\mathbf{w}^T \mathbf{x}_n) - y_n |$  is an NP-hard problem!
- Can we do better? **Yes... soft thresholds to the rescue!**



# Logistic function, odds, and logits

- Logistic function

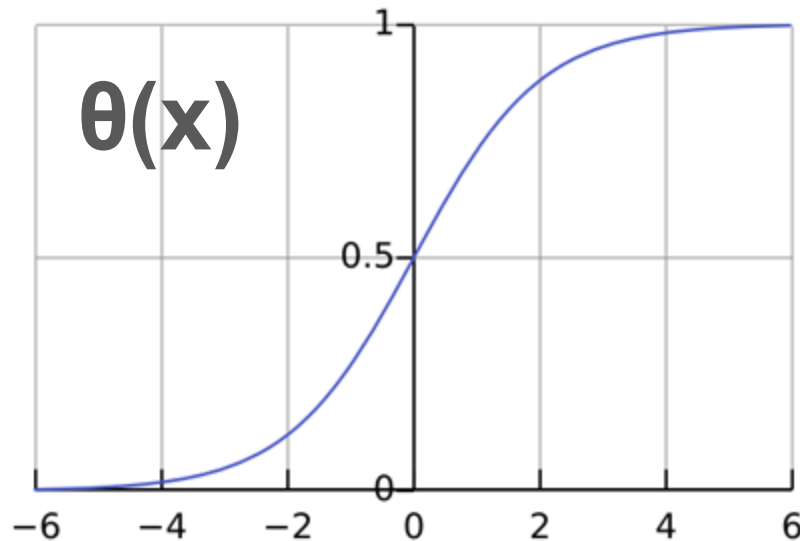
$$\theta(x) = \frac{\exp(x)}{1 + \exp(x)}$$

- Odds:  $\exp(x) = \frac{\theta(x)}{1 - \theta(x)}$

- Logits:

$$\text{logit}(\theta) = x = \ln \left( \frac{\theta(x)}{1 - \theta(x)} \right)$$

Used to model probabilities  
(also called Sigmoid function)



# Properties of the logistic function

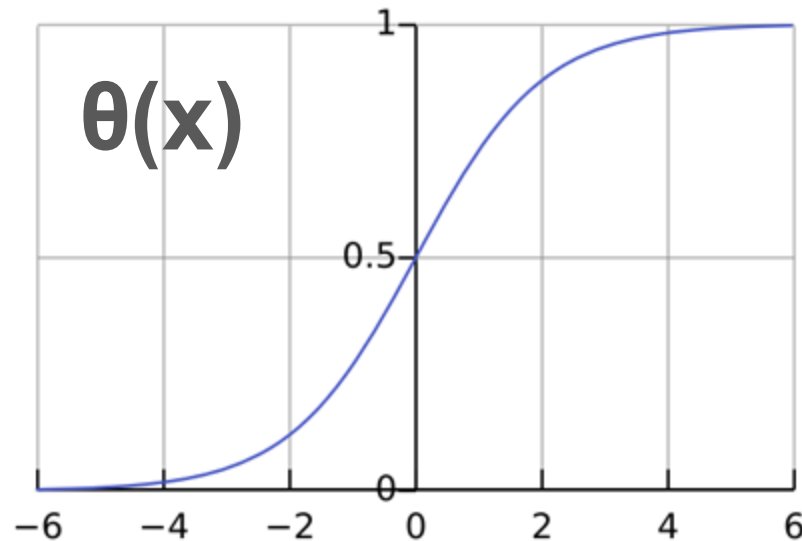
$$\theta(x) = \frac{\exp(x)}{1 + \exp(x)}$$

$$\theta(x) = \frac{1}{1 + \exp(-x)}$$

$$\theta(-x) = 1 - \theta(x) = \frac{1}{1 + \exp(x)}$$

$$\frac{d\theta(x)}{dx} = \theta(x) (1 - \theta(x)) = \theta(x) \theta(-x)$$

$$\theta(-\infty) = 0, \theta(0) = \frac{1}{2}, \theta(\infty) = 1$$



# Logistic regression (binary classification)

- Estimates the probability of  $y_n$  belonging to a given category conditioned to the value of  $\mathbf{x}_n$  by using a logistic function:

$$p(y_n = 1 \mid \mathbf{x}_n) = \theta(\mathbf{w}^T \mathbf{x}_n)$$

- Accordingly, the probability of  $y_n$  not belonging to the given category is given by:

$$p(y_n = -1 \mid \mathbf{x}_n) = 1 - \theta(\mathbf{w}^T \mathbf{x}_n) = \theta(-\mathbf{w}^T \mathbf{x}_n)$$

- From where we can write  $p(y_n \mid \mathbf{x}_n) = \theta(y_n \mathbf{w}^T \mathbf{x}_n)$

# Maximum likelihood and error function

- Find the weights  $\mathbf{w}$  that maximize the likelihood of the observed in-sample data points (maximum likelihood):

$$L(\mathbf{w}) = \prod_N p(y_n | \mathbf{x}_n) = \prod_N \theta(y_n \mathbf{w}^T \mathbf{x}_n)$$

- Alternatively, minimize an error function (negative log):

$$\begin{aligned} E(\mathbf{w}) &= -1/N \ln( \prod_N p(y_n | \mathbf{x}_n) ) = 1/N \sum_N \ln( 1 / \theta(y_n \mathbf{w}^T \mathbf{x}_n) ) \\ &= 1/N \sum_N \ln( 1 + \exp( - y_n \mathbf{w}^T \mathbf{x}_n ) ) \end{aligned}$$

# Gradient of the error and error minimization

- The gradient of the error can be computed:

$$\nabla E(\mathbf{w}) = -1/N \sum_N \frac{y_n \mathbf{x}_n}{1 + \exp(y_n \mathbf{w}^T \mathbf{x}_n)}$$

- However, when making  $\nabla E(\mathbf{w}) = 0$ , it is not possible to obtain a closed solution for the weight vector  $\mathbf{w}$
- An recursive procedure to compute  $\mathbf{w}$  is needed



## Gradient descent algorithm

- Start with an initial set of weights  $\mathbf{w}(t)$  at time step  $t=0$
- Compute the gradient:

$$\nabla E(\mathbf{w}(t)) = -1/N \sum_N \frac{y_n \mathbf{x}_n}{1 + \exp(y_n \mathbf{w}^T(t) \mathbf{x}_n)}$$

- Update the weights moving in the opposite gradient direction:

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla E(\mathbf{w}(t))$$

- Keep iterating until termination criteria is satisfied

# Some practical considerations on gradient descent

- Initialization:  $w(t=0)$  randomly generated (normal distribution with zero mean and small variance)
- Termination: a combination of termination criteria
  - A maximum number of iterations
  - Marginal error improvement (i.e. small gradient)
  - Small error value over a few iterations
- Computing the error over the whole in-sample dataset (batch gradient descent) might be computationally expensive!



# Stochastic gradient descent algorithm

- Start with an initial set of weights  $\mathbf{w}(t)$  at time step  $t=0$
- Randomly select a single data point and consider its error only:

$$\nabla e_n(\mathbf{w}(t)) = \frac{y_n \mathbf{x}_n}{1 + \exp(y_n \mathbf{w}^T(t) \mathbf{x}_n)}$$

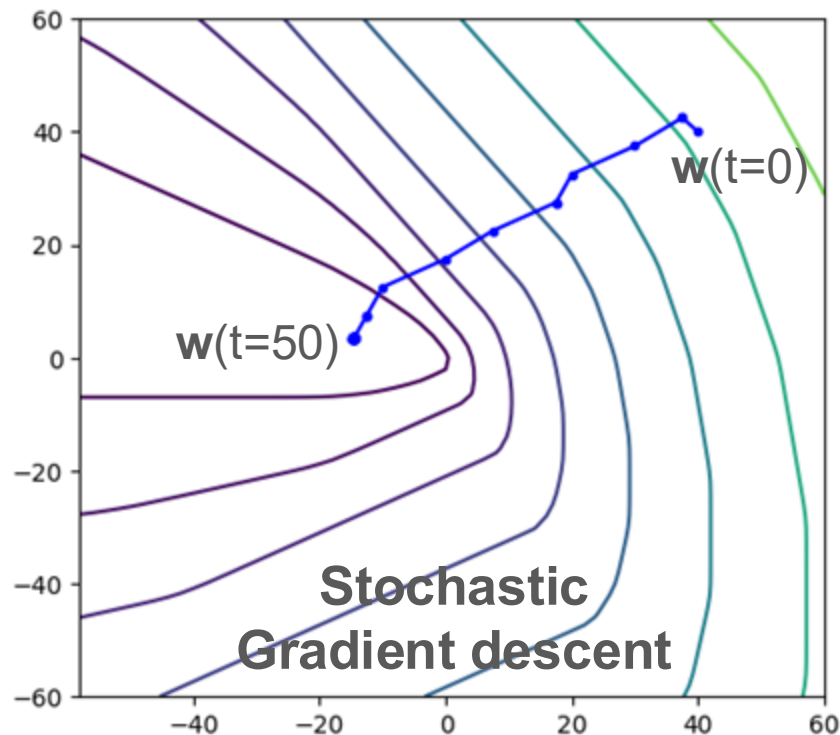
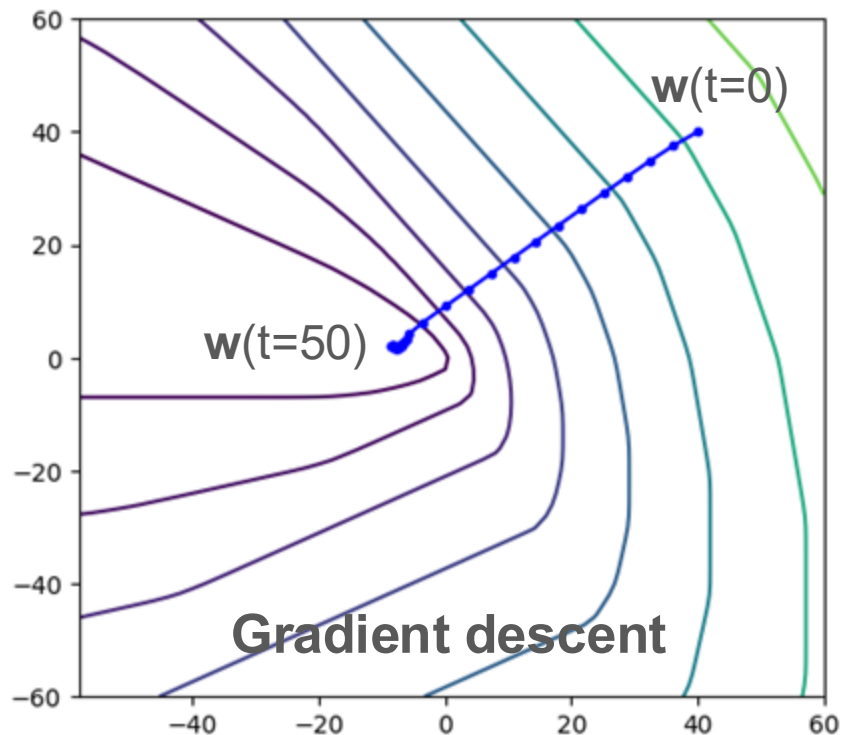
- Update the weights moving in the opposite gradient direction:

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \boldsymbol{\eta} \nabla e_n(\mathbf{w}(t))$$

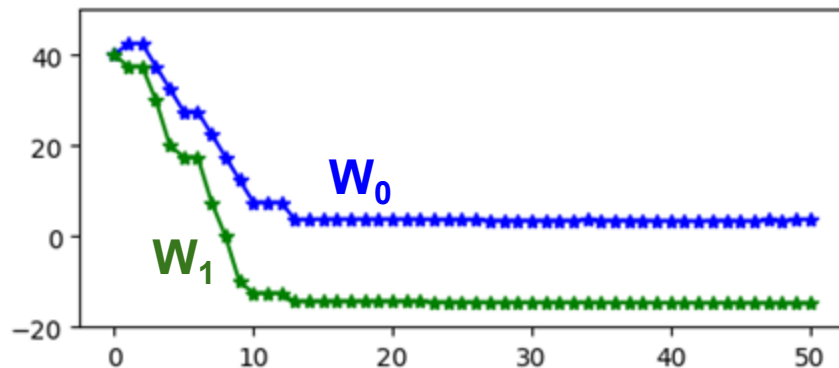
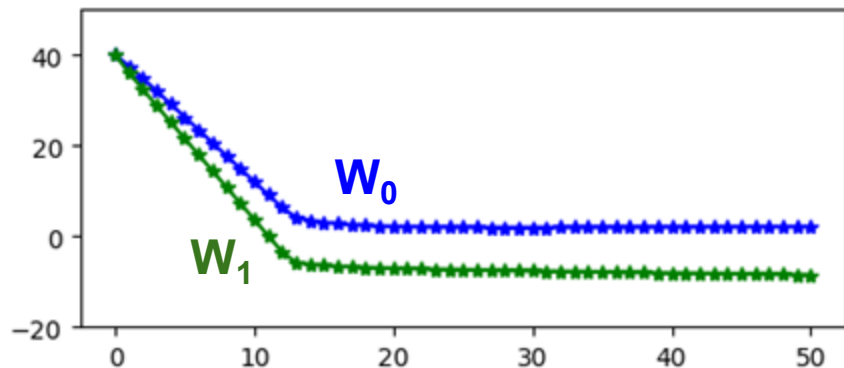
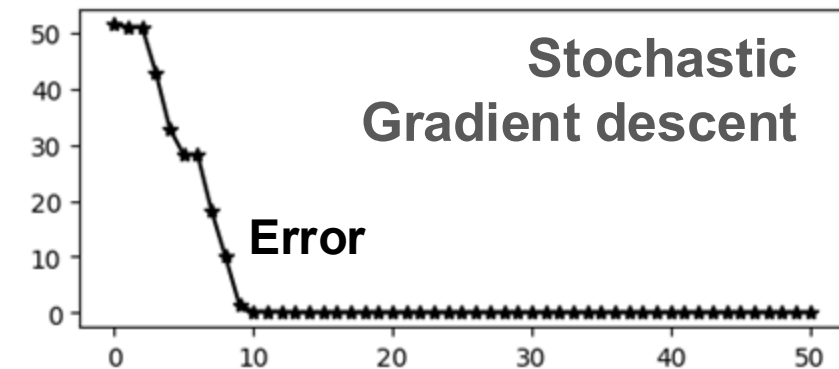
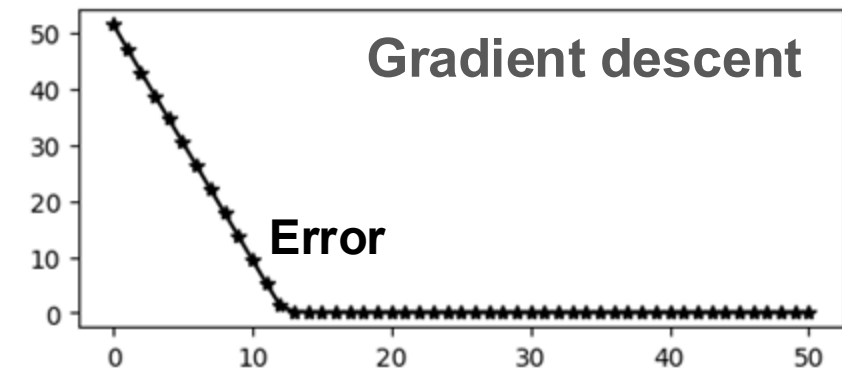
- Keep iterating until termination criteria is satisfied



# Gradient descent vs. stochastic gradient descent



# Gradient descent vs. stochastic gradient descent



# Multilinear logistic regression with softmax

- The presented logistic regression approach can be extended to the case of  $K > 2$  classes
- Using softmax coding, the probability of each class is represented as:

$$p(y_n = k \mid \mathbf{x}_n) = \frac{\exp(\mathbf{w}_k^T \mathbf{x}_n)}{\sum_{i=1}^K \exp(\mathbf{w}_i^T \mathbf{x}_n)} \quad \text{for } k = 1, 2, 3 \dots K$$

- Where  $\mathbf{w}_k$  is the vector of  $M$  weights  $w_{mk}$  for class  $k$  (i.e. a total number of  $M \times K$  weights need to be estimated)



# The Bayes theorem

- Joint probabilities:  $P(A,B) = P(B,A)$
- Using conditional probabilities:  $P(B) P(A|B) = P(A) P(B|A)$
- Bayes theorem: 
$$\underset{\text{posterior}}{P(A|B)} = \underset{\text{prior}}{P(A)} \underset{\text{likelihood}}{P(B|A)} / \underset{\text{evidence}}{P(B)}$$
- $P(B)$  can also be stated as  $P(B) = P(A) P(B|A) + P(\sim A) P(B|\sim A)$
- And more generically:  $P(B) = \sum_a P(A=a) P(B|A=a)$

# The naive Bayes classifier

- Following a similar setting to the one used in multinomial logistic regression, by using Bayes theorem, we can propose:

$$p(y_n = k \mid \mathbf{x}_n) = \frac{p(y_n = k) p(\mathbf{x}_n \mid y_n = k)}{\sum_{i=1}^K p(y_n = i) p(\mathbf{x}_n \mid y_n = i)} \quad \text{for } k = 1, 2, 3 \dots K$$

- Where we need to estimate the priors and likelihoods

**VERY EASY**

**VERY DIFFICULT**



# The naive Bayes assumption

- Conditional independence assumption: within each class, the predictors (conditional probabilities) are independent

$$p(\mathbf{x}_n \mid y_n = k) = \prod_M p(x_{nm} \mid y_n = k)$$

- Priors  $p(y_n = k)$  can be estimated by computing the fraction of the training data that belongs to each class  $k$
- Likelihoods  $p(x_{nm} \mid y_n = k)$  can be estimated by approximating  $p(x)$  distributions within each class  $k$

# Maximum likelihood

- In the same way it was done with logistic regression, model parameters can be also estimated from in-sample data
- Find parameters that maximize the likelihood of the observed in-sample data points (i.e. minimize the negative log of it)
- Likelihoods can be modeled either by assuming a distribution (gaussian, multinomial, bernoulli, etc.) or by means of non-parametric models



## Log-likelihood ratios

- As seen before, the probability of a class  $y_n = k$  is:

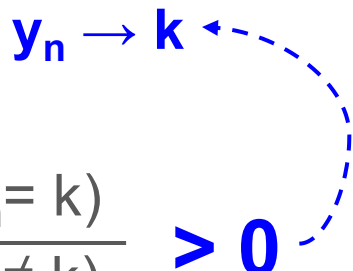
$$p(y_n = k \mid \mathbf{x}_n) = p(y_n = k) \prod_M p(x_{nm} \mid y_n = k) / p(\mathbf{x}_n)$$

- And the probability of the non-class  $y_n \neq k$  is:

$$p(y_n \neq k \mid \mathbf{x}_n) = p(y_n \neq k) \prod_M p(x_{nm} \mid y_n \neq k) / p(\mathbf{x}_n)$$

- By dividing and taking the logarithm:

$$\ln \frac{p(y_n = k \mid \mathbf{x}_n)}{p(y_n \neq k \mid \mathbf{x}_n)} = \ln \frac{p(y_n = k)}{p(y_n \neq k)} + \sum_M \ln \frac{p(x_{nm} \mid y_n = k)}{p(x_{nm} \mid y_n \neq k)} > 0$$







# A naive Bayes classification example

- Consider the document collection represented in the table
- Use log-likelihood ratios to classify of the unlabeled document

| Document | Category | Word-1 | Word-2 | Word-3 | Word-4 | Word-5 | Total Words |
|----------|----------|--------|--------|--------|--------|--------|-------------|
| D1       | A        | 3      | 1      | 2      | 2      | 1      | 9           |
| D2       | A        | 2      | 0      | 3      | 2      | 0      | 7           |
| D3       | B        | 2      | 3      | 1      | 2      | 2      | 10          |
| D4       | A        | 1      | 0      | 2      | 3      | 1      | 7           |
| D5       | B        | 1      | 3      | 1      | 1      | 2      | 8           |
| D6       | A        | 2      | 1      | 1      | 3      | 2      | 9           |
| D7       | ?        | 0      | 2      | 1      | 1      | 2      | 6           |



## A naive Bayes classification example

- Let us start computing the priors  $P(C=A)$  and  $P(C=B)$

$$P(C=A) = \#Docs(C=A) / \#Docs = 4/6 = 2/3$$

$$P(C=B) = \#Docs(C=B) / \#Docs = 2/6 = 1/3$$

- Next we need to estimate the likelihoods. By using the conditional independence assumption:

$$P(Doc_n | C_n = K) = \prod_M p(w_{nm} | C_n = K)$$



# A naive Bayes classification example

- Consider the documents in category A, from which we can estimate conditional probabilities of the form  $P(w_m | C_n = A)$

| Document     | Category | Word-1 | Word-2 | Word-3 | Word-4 | Word-5 | Total Words |
|--------------|----------|--------|--------|--------|--------|--------|-------------|
| D1           | A        | 3      | 1      | 2      | 2      | 1      | 9           |
| D2           | A        | 2      | 0      | 3      | 2      | 0      | 7           |
| D4           | A        | 1      | 0      | 2      | 3      | 1      | 7           |
| D6           | A        | 2      | 1      | 1      | 3      | 2      | 9           |
| Total Counts |          | 8      | 2      | 8      | 10     | 4      | 32          |
| $p(w_m   A)$ |          | 1/4    | 1/16   | 1/4    | 5/16   | 1/8    | -           |



## A naive Bayes classification example

- Similarly, consider the documents in category B, from which we can estimate conditional probabilities of the form  $P(w_m | C_n = B)$

| Document     | Category | Word-1 | Word-2 | Word-3 | Word-4 | Word-5 | Total Words |
|--------------|----------|--------|--------|--------|--------|--------|-------------|
| D3           | B        | 2      | 3      | 1      | 2      | 2      | 10          |
| D5           | B        | 1      | 3      | 1      | 1      | 2      | 8           |
| Total Counts |          | 3      | 6      | 2      | 3      | 4      | 18          |
| $p(w_m A)$   |          | 1/6    | 1/3    | 1/9    | 1/6    | 2/9    | -           |



# A naive Bayes classification example

- These are the resulting model parameters

| Category | $p(c)$ | $p(w_1 c)$ | $p(w_2 c)$ | $p(w_3 c)$ | $p(w_4 c)$ | $p(w_5 c)$ |
|----------|--------|------------|------------|------------|------------|------------|
| A        | $2/3$  | $1/4$      | $1/16$     | $1/4$      | $5/16$     | $1/8$      |
| B        | $1/3$  | $1/6$      | $1/3$      | $1/9$      | $1/6$      | $2/9$      |

- Which can be used to determine the category of document  $D_7$

| Document | Category | Word-1 | Word-2 | Word-3 | Word-4 | Word-5 | Total Words |
|----------|----------|--------|--------|--------|--------|--------|-------------|
| D7       | ?        | 0      | 2      | 1      | 1      | 2      | 6           |



## A naive Bayes classification example

$$\ln \frac{p(A | D_7)}{p(B | D_7)} = \ln \frac{p(A)}{p(B)} + \sum_M \ln \frac{p(w_{7m} | A)}{p(w_{7m} | B)}$$

$$= \ln(2) + 0 \times \ln(6/4) + 2 \times \ln(3/16) + 1 \times \ln(9/4) + 1 \times \ln(30/16) + 2 \times \ln(9/16)$$

$$= 0.6931 + 0 - 3.3479 + 0.8109 + 0.6286 - 1.1507 = -2.366 < 0$$

**$\Rightarrow$  D7 belongs to category B**



# Classification metrics...

$$\text{F1-score} = 2 \times \frac{\text{Pre} \times \text{Rec}}{\text{Pre} + \text{Rec}}$$

**Confusion  
Matrix**

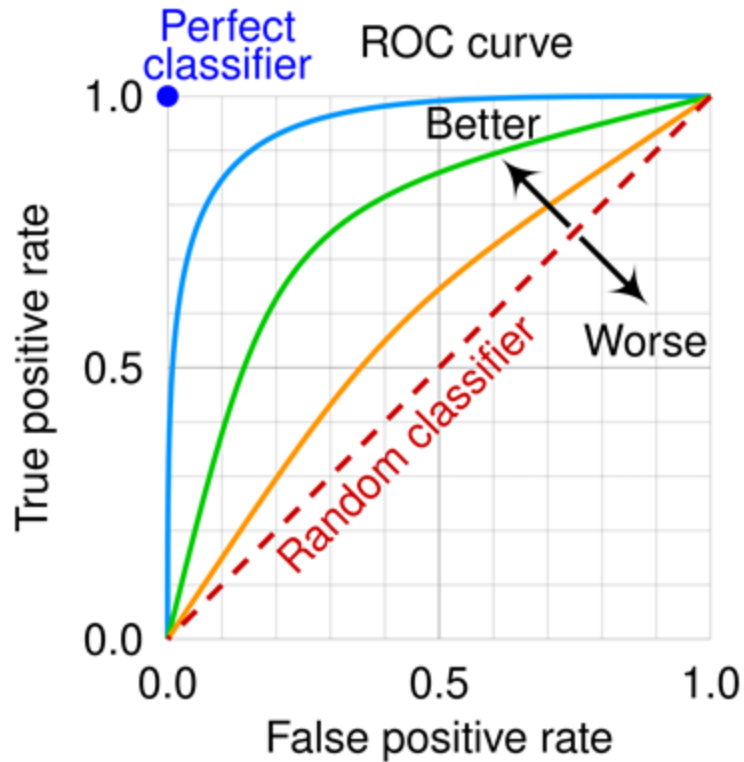
**PREDICTIONS**

| ACTUAL    |   |    |            |            | Recall     |                                               |
|-----------|---|----|------------|------------|------------|-----------------------------------------------|
|           |   | A  | B          | C          |            |                                               |
|           | A | AA | AB         | AC         | AA/(AB+AC) | Rec(A)                                        |
|           | B | BA | BB         | BC         | BB/(BA+BC) | Rec(B)                                        |
|           | C | CA | CB         | CC         | CC/(CA+CB) | Rec(C)                                        |
| Precision |   |    | AA/(BA+CA) | BB/(AB+CB) | CC/(AC+BC) | Accuracy                                      |
|           |   |    | Pre(A)     | Pre(B)     | Pre(C)     | $\frac{AA+BB+CC}{AA+AB+AC+BA+BB+BC+CA+CB+CC}$ |



# Classification metrics...

- Receiver Operating Characteristic (ROC) curve:
  - Graphic representation of a binary classifier performance
  - True Positive Rate (TPR) vs False Positive Rate (FPR)
- Area Under the Curve (AUC)
  - Perfect classifier:  $AUC = 1.0$
  - Random classifier:  $AUC = 0.5$







**Hands-on Coding Session W3S1**

# Hands on coding session W3S1



- Get familiar with the notebook for W3S1 and its contents
- Look at the documentation for some of the functions used there from python modules:
  - Statsmodels: <https://www.statsmodels.org/stable/index.html>
  - Scikit-Learn: <https://scikit-learn.org/0.16/index.html#>
- Work on exercises in Part 1 (linear and multilinear regression)
- Work on exercises in Part 2 (logistic regression classification)
- Work on exercises in Part 3 (naive Bayes classification)

## **Week 3 - Closing remarks**

### ***Wrap-up and Homework 3***

- What did we look at today?
- Further reading and support materials
- Third homework assignment

# What did we look at today?

|                                     |                                                                                                                                                                                  |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Linear Regression</b>            | <ul style="list-style-type: none"><li>• Simple Linear Regression</li><li>• Multiple Linear Regression</li><li>• Linear Model Extensions</li></ul>                                |
| <b>Linear Classification</b>        | <ul style="list-style-type: none"><li>• Logistic Regression</li><li>• Gradient Descent Algorithm</li><li>• The Naive Bayes Classifier</li></ul>                                  |
| <b>Hands-on coding session W3S1</b> | <ul style="list-style-type: none"><li>• Linear and Multilinear Regressions</li><li>• Classification with Logistic Regression</li><li>• Classification with Naive Bayes</li></ul> |

## Further reading and support materials

- James, Witten, Hastie, Tibshirani, Taylor, “An Introduction to Statistical Learning with Applications in Python” (Chapters 3 and 4) <https://www.statlearning.com/>
- Abu-Mostafa, Magdon-Ismael, Lin, “Learning From Data” (Chapter 3) <https://work.caltech.edu/telecourse>

## Third homework assignment (due date 07/17)

- From the book “An Introduction to Statistical Learning with Applications in Python” read the following sections and be ready to answer questions about them:
  - 3.3.3 “Linear Regression: Potential Problems”
  - 4.4.1&2 “Classification: Linear Discriminant Analysis”
  - 4.4.3 “Classification: Quadratic Discriminant Analysis”
- Complete all coding exercises from coding session W3S1
- Work with your project team to give a title to your project!



## **Remember:**

Office Hours

Tuesday 07/15 5:30-6:30pm

Next Class

Thursday 07/17, 5:30-9:30pm

Third Homework Due

Thursday 07/17, 11:59pm